LAB 05 — Sequence Models: RNN, LSTM and GRU

Môn học: Xử lý ngôn ngữ tự nhiên và ứng dụng | **Học kỳ:** Học kỳ I - 2026

Giảng viên / TA: Phạm Ngọc Hải ▪ **Hạn nộp:** 23:59 - 21/10/2026

Chủ đề: From Static Representations to Sequential Modeling

1. Bối cảnh

Ở LAB 04, sinh viên đã sử dụng TF-IDF và Average Word2Vec để đưa văn bản vào bài toán classification.

Tuy nhiên, hai representation này bộc lộ một vấn đề quan trọng:

> **Văn bản là một chuỗi có thứ tự, nhưng representation tĩnh không mô hình hóa thứ tự một cách trực tiếp.**

Ví dụ:

```
S1 = "this movie is good"
S2 = "this movie is not good"
```

Average Word2Vec biểu diễn câu bằng:

$$\vec{x} = \frac{1}{n}\sum_{i=1}^{n}\vec{w}_i.$$

Do đó, thông tin về thứ tự từ bị mất.

LAB 05 chuyển sang một cách tiếp cận khác:

```
Text
 ↓
Token sequence
 ↓
Word embeddings
 ↓
RNN / LSTM / GRU
 ↓
Hidden state
 ↓
Classifier
 ↓
Prediction
```

Thay vì biểu diễn toàn bộ câu bằng một vector cố định ngay từ đầu, model đọc sequence từng bước và cập nhật trạng thái theo thời gian.

Mục tiêu không chỉ là:

> "LSTM có accuracy cao hơn Logistic Regression hay không?"

Mà là:

> **Tại sao sequence model có thể sử dụng thông tin về thứ tự và context, và tại sao RNN vẫn gặp vấn đề khi sequence dài?**

LAB 05 vì vậy là cầu nối trực tiếp từ các representation ở LAB 01–04 tới Attention và Transformer.

---

2. Mục tiêu học tập

Sau khi hoàn thành lab, sinh viên có thể:

- **LO1:** Giải thích sequence modeling trong NLP.
- **LO2:** Mô tả cơ chế recurrent computation của RNN.
- **LO3:** Tính một bước forward của RNN bằng tay.
- **LO4:** Giải thích hidden state và information flow qua sequence.
- **LO5:** Nhận diện vanishing gradient và exploding gradient.
- **LO6:** Giải thích motivation của LSTM và các gates.
- **LO7:** Mô tả sự khác nhau giữa RNN, LSTM và GRU.
- **LO8:** Xây dựng sequence classification pipeline.
- **LO9:** So sánh model không có sequence modeling với RNN/LSTM/GRU.
- **LO10:** Phân tích ảnh hưởng của sequence length và padding/truncation.
- **LO11:** Phân tích lỗi của sequence classifier.
- **LO12:** Nhận diện giới hạn của recurrent models và động lực dẫn tới attention.

---

3. Cấu trúc lab

Giai đoạn	Thời lượng	Hoạt động
Theory recap	20 phút	Sequence modeling và RNN
Calculation	25 phút	RNN forward computation
Prediction	10 phút	Dự đoán behavior của sequence models
Experiment 1	20 phút	Tokenization, sequence length
Core implementation	35 phút	RNN cell từ đầu
Experiment 2	30 phút	RNN vs LSTM vs GRU
Evaluation	25 phút	Classification metrics
Experiment 3	25 phút	Sequence length / truncation
Error analysis	25 phút	Context và long-range dependency
Reflection	15 phút	RNN → Attention

---

4. Theory recap

4.1 Từ document representation sang sequence

Ở LAB 04:

```
Document
 ↓
Fixed-size vector
 ↓
Classifier
```

Ví dụ với Average Word2Vec:

$$\vec{x} = \frac{1}{T}\sum_{t=1}^{T}\vec{w}_t.$$

Trong sequence model:

```
w1 → w2 → w3 → ... → wT
     ↓    ↓          ↓
    h1   h2   ...   hT
```

Mỗi token được xử lý theo thứ tự.

Hidden state:

$$h_t = f(x_t, h_{t-1}).$$

Do đó:

$$h_t$$

mang thông tin từ các token trước đó.

---

5. RNN

Một RNN đơn giản có thể được viết:

$$h_t = \tanh\left(W_{xh}x_t + W_{hh}h_{t-1} + b_h\right).$$

Trong đó:

- (x_t): embedding của token tại thời điểm (t);
- (h_{t-1}): hidden state trước đó;
- (h_t): hidden state hiện tại;
- (W_{xh}): input-to-hidden weights;
- (W_{hh}): recurrent weights;
- (b_h): bias.

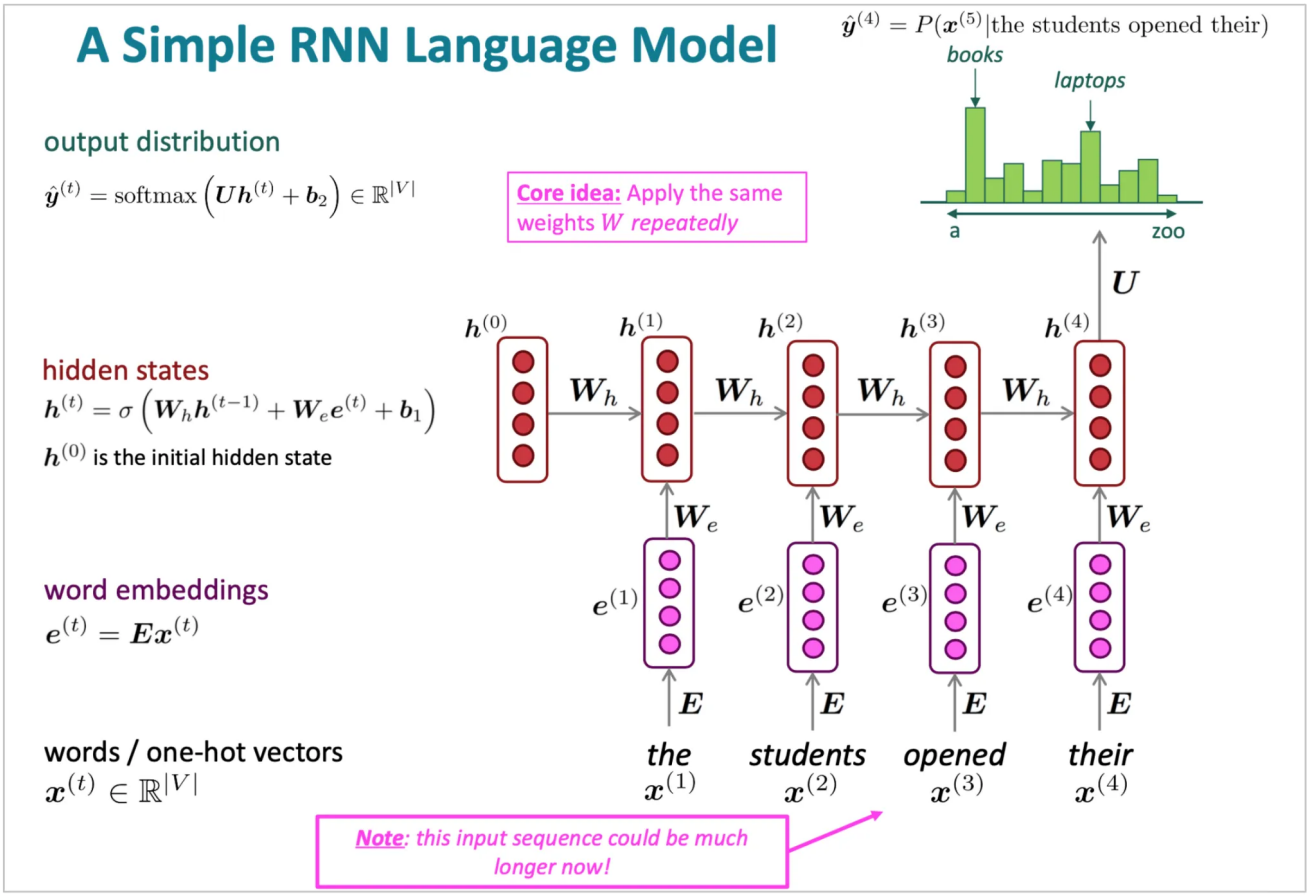
Output classification có thể được tính:

$$z = W_{hy}h_T + b_y.$$

Sau đó:

$$P(y|x) = \text{softmax}(z).$$

Với binary classification có thể sử dụng sigmoid.



Hình 1: Kiến trúc 1 RNN đơn giản

---

6. Bài tập tính toán

Bài 1 — Một bước RNN

Cho:

$$x_t = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad h_{t-1} = \begin{bmatrix} 0.5 \\ 0.2 \end{bmatrix}.$$

Giả sử:

$$W_{xh} = \begin{bmatrix} 0.5 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}$$

và:

$$W_{hh} = \begin{bmatrix} 0.3 & 0.1 \\ 0.2 & 0.5 \end{bmatrix}.$$

Bias:

$$b_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Tính:

$$a_t = W_{xh}x_t + W_{hh}h_{t-1}.$$

Sau đó tính:

$$h_t = \tanh(a_t).$$

Sinh viên phải trình bày đầy đủ từng bước.

---

7. Bài 2 — Information flow

Cho sequence:

```
"I did not like this movie"
```

Giả sử model đọc từ trái sang phải.

Sinh viên mô tả hidden state sau từng bước:

```
"I"
 ↓
"did"
 ↓
"not"
 ↓
"like"
 ↓
"this"
 ↓
"movie"
```

Trả lời:

> Tại thời điểm model đọc `"like"`, thông tin `"not"` có thể ảnh hưởng tới hidden state như thế nào?

Sau đó so sánh với Average Word2Vec.

---

8. Bài 3 — Sequence order

Cho hai sequence:

```
S1 = "dog bites man"

S2 = "man bites dog"
```

Giả sử cả hai câu có cùng vocabulary.

Trả lời:

> Average Word2Vec có thể phân biệt hai câu này dựa trên thứ tự hay không?

Sau đó:

RNN có khả năng phân biệt chúng dựa trên thứ tự như thế nào?

Sinh viên phải trả lời dựa trên cơ chế model, không chỉ dựa trên kết quả thực nghiệm.

---

9. Vanishing gradient

Trong RNN, hidden state được truyền qua nhiều timestep:

$$h_1 \to h_2 \to h_3 \to \cdots \to h_T.$$

Khi backpropagation through time, gradient được truyền ngược qua nhiều timestep.

Về trực giác:

$$\frac{\partial L}{\partial h_t}$$

có thể chứa tích của nhiều Jacobian:

$$\frac{\partial h_T}{\partial h_t} = \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}.$$

Nếu các thành phần trong tích có độ lớn nhỏ hơn 1, gradient có thể giảm rất nhanh:

$$\left| \prod_k a_k \right| \to 0.$$

Đây là **vanishing gradient**.

Ngược lại, nếu các thành phần quá lớn, gradient có thể tăng rất nhanh:

$$\left| \prod_k a_k \right| \to \infty.$$

Đây là **exploding gradient**.

Sinh viên không cần chứng minh đầy đủ về mặt giải tích, nhưng phải giải thích được:

Vì sao sequence càng dài có thể khiến RNN khó học dependency ở xa?

---

10. Prediction trước experiment

Trước khi train bất kỳ model nào, sinh viên ghi:

```
Prediction:
Reason:
Confidence:
```

Prediction 1

Giữa:

```
TF-IDF + Logistic Regression
```

và:

```
RNN + Embedding
```

model nào có khả năng xử lý word order trực tiếp hơn?

Giải thích.

Prediction 2

Giữa:

```
RNN
LSTM
```

model nào có khả năng xử lý dependency dài tốt hơn?

Giải thích bằng cơ chế model, không chỉ nói "LSTM hiện đại hơn".

Prediction 3

Nếu sequence length tăng từ:

```
50 → 200
```

training của RNN sẽ thay đổi như thế nào?

Prediction 4

Nếu truncate toàn bộ review về 30 tokens, thông tin classification có thể bị ảnh hưởng không?

Prediction 5

LSTM có chắc chắn cho accuracy cao hơn RNN trên mọi dataset không?

---

11. Experiment 1 — Dataset inspection

Dataset chính:

IMDb Movie Reviews

Task:

```
positive
negative
```

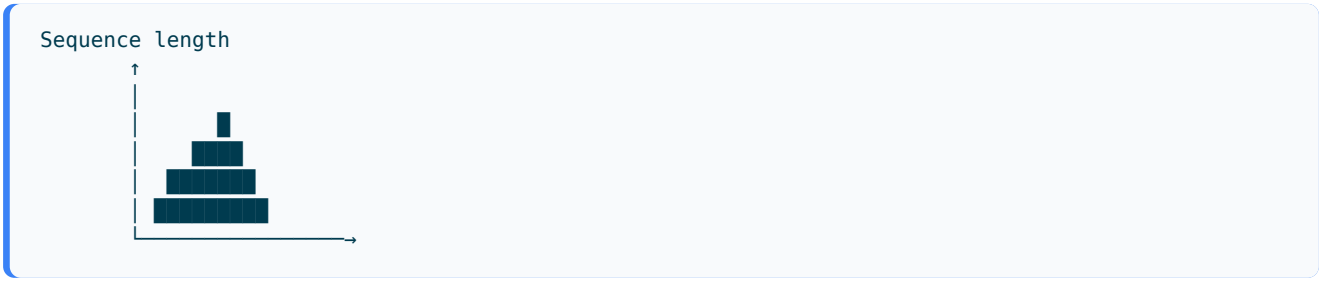
IMDb được chọn vì phù hợp với sequence classification và cho phép tiếp tục sử dụng sentiment classification từ LAB 04 nhưng thay đổi cách representation/modeling.

Sinh viên phải kiểm tra:

- số samples;
- số classes;
- class distribution;
- document length;
- minimum sequence length;
- median sequence length;

- mean sequence length;
- maximum sequence length;
- duplicate documents;
- missing text;
- missing labels.

Đặc biệt phải vẽ histogram:



Trả lời:

> Nếu phải chọn `max_length`, nên chọn dựa trên thông tin nào?

Không chấp nhận câu trả lời chỉ là:

> "Chọn 100 vì model chạy nhanh."

---

12. Tokenization và vocabulary

Pipeline:

```
Raw text
 ↓
Tokenizer
 ↓
Token IDs
 ↓
Padding / truncation
 ↓
Embedding
 ↓
RNN/LSTM/GRU
```

Ví dụ:

```
"I really like this movie"
```

có thể trở thành:

```
[12, 47, 31, 8, 102]
```

Sau padding:

```
[12, 47, 31, 8, 102, 0, 0, 0]
```

Trong đó:

```
0 = PAD
```

Sinh viên phải hiểu:

> Padding không phải là thông tin ngôn ngữ thực sự.

Cần phân biệt:

```
padding
```

và:

```
truncation
```

---

13. Experiment 2 — Padding và truncation

Thử ít nhất ba giá trị:

```
max_length = 50
max_length = 100
max_length = 200
```

Ghi:

Max length	% tokens retained	Training time	Accuracy	F1
50				
100				
200				

Trả lời:

> Tăng `max_length` có luôn cải thiện performance không?

Và:

> Nếu `max_length` quá nhỏ, loại thông tin nào có thể bị mất?

---

14. Core implementation — Simple RNN cell

Sinh viên không cần tự xây dựng toàn bộ RNN training framework.

Thay vào đó, phải tự implement **một RNN cell** để hiểu cơ chế.

Yêu cầu:

```
class SimpleRNNCell:
    def __init__(self, input_size, hidden_size):
        ...

    def forward(self, x_t, h_prev):
        ...
```

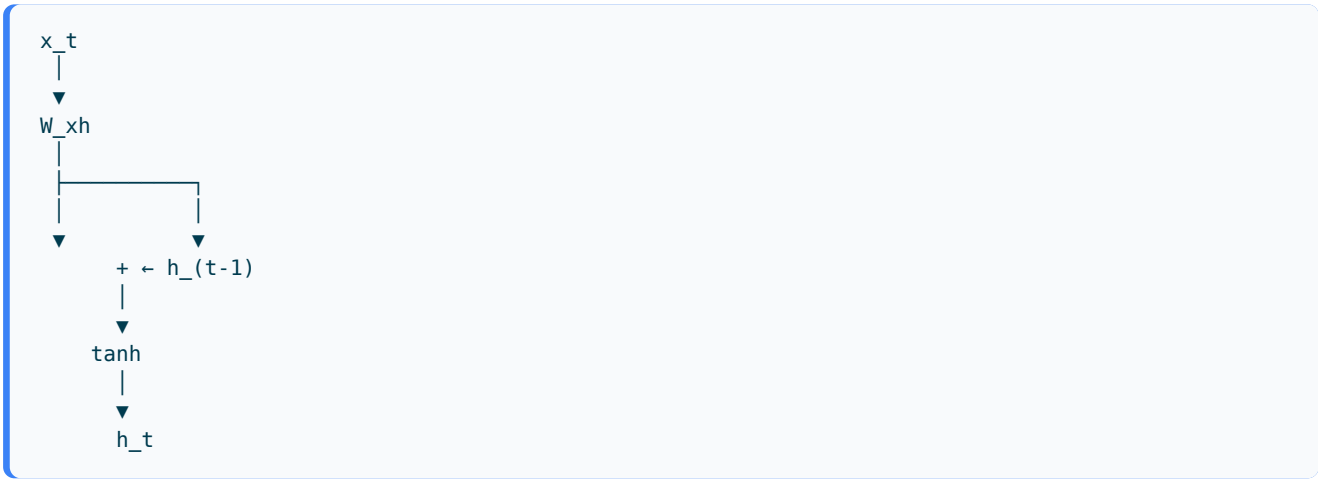
Thực hiện:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h).$$

Test bằng một input nhỏ.

Sau đó kiểm tra kết quả với implementation tương ứng của framework.

Sinh viên phải giải thích:

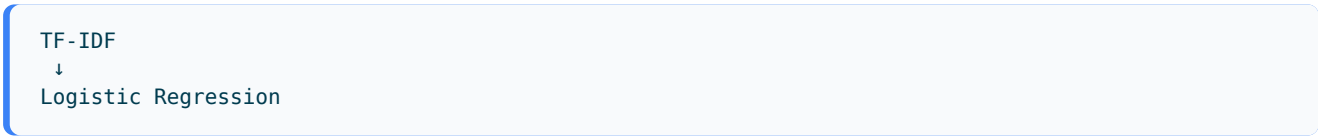


Mục đích của phần này là hiểu computation graph, không phải tự xây dựng PyTorch.

---

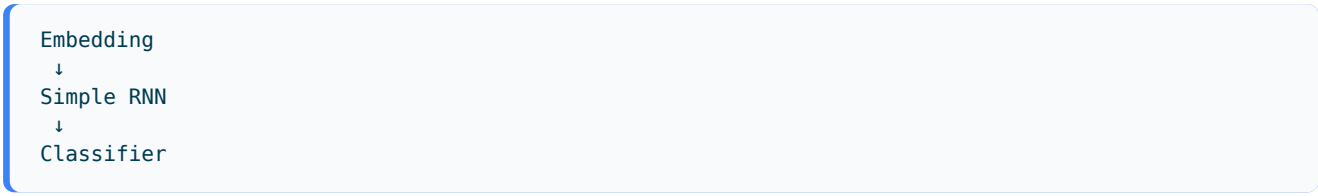
15. Experiment 3 — Baseline

Sử dụng baseline từ LAB 04:



Kết quả được lấy làm baseline.

Sau đó xây dựng:



So sánh:

Model	Representation	Accuracy	Precision	Recall	F1	Training time
Logistic Regression	TF-IDF					
RNN	Embedding					

Câu hỏi:

RNN có thực sự sử dụng thông tin sequence mà TF-IDF không sử dụng hay không?

Sinh viên phải đưa ra bằng chứng từ experiment.

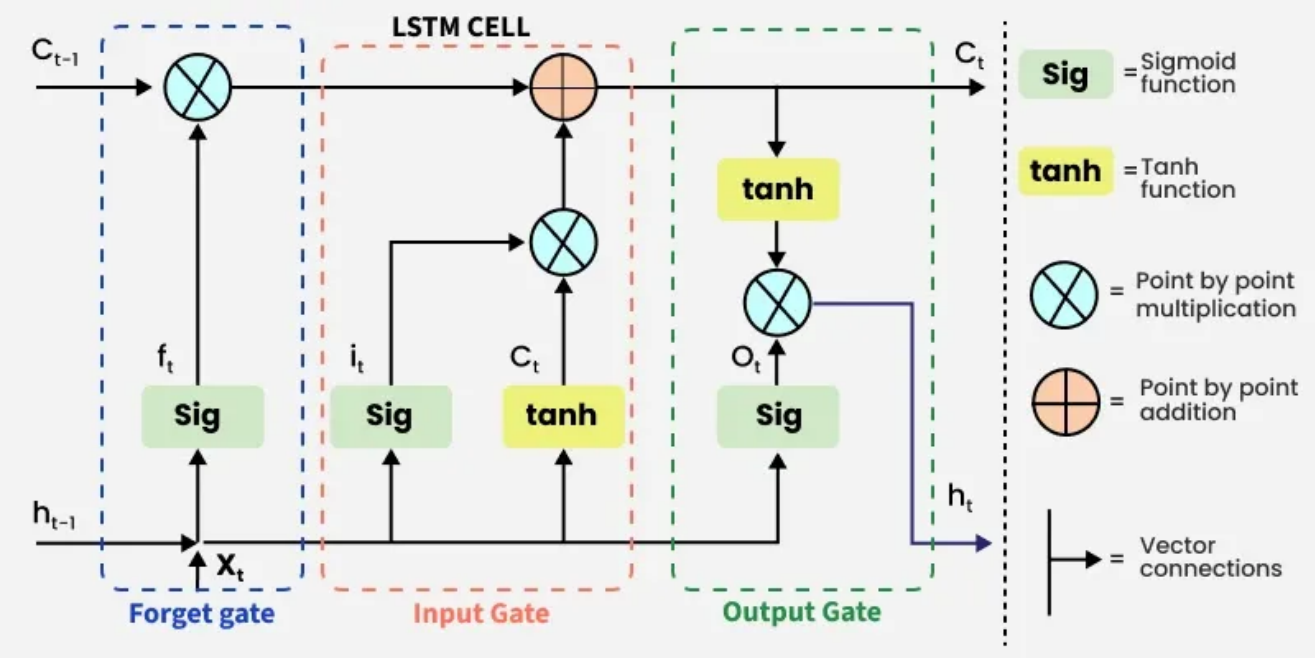
---

16. LSTM

RNN gặp khó khăn với dependency dài.

LSTM đưa vào một **cell state** và các **gates** để kiểm soát information flow.

Một cách biểu diễn đơn giản:



Hình 2: Sơ đồ kiến trúc LSTM Gate

Forget gate:

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$$

Input gate:

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i)$$

Candidate state:

$$\tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c).$$

Cell state:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t.$$

Output gate:

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o)$$

Hidden state:

$$h_t = o_t \odot \tanh(c_t).$$

Sinh viên không cần nhớ tất cả công thức bằng máy móc.

Điều quan trọng là hiểu:

```
Forget gate
      ↓
Input gate
      ↓
```

```
Cell state
      ↓
Output gate
      ↓
Hidden state
```

---

17. Bài tập tính toán — LSTM gates

Cho một timestep với:

```
forget gate = 0.9
input gate  = 0.2
candidate   = 0.5
cell state previous = 0.8
```

Tính:

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t.$$

Sau đó trả lời:

> Nếu forget gate gần 1, model có xu hướng giữ hay loại bỏ thông tin cũ?

Và:

> Nếu forget gate gần 0 thì sao?

Mục tiêu của bài này là hiểu **gate như một cơ chế điều khiển information flow**, không phải thuộc công thức.

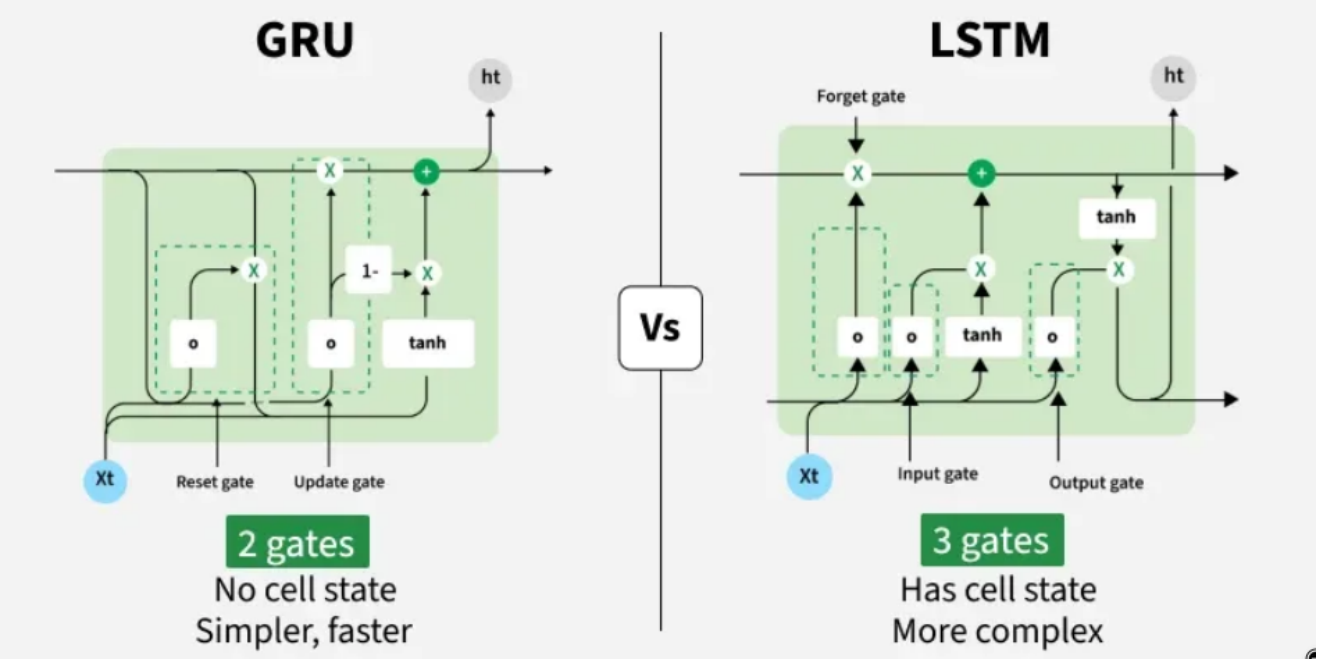
---

18. GRU

GRU là một recurrent architecture khác với cấu trúc đơn giản hơn LSTM.

Sinh viên cần nắm:

- GRU sử dụng hidden state;
- GRU có update mechanism;
- GRU không có cell state riêng như LSTM;
- GRU thường có ít tham số hơn LSTM.



Hình 3: Kiến trúc GRU so vs Kiến trúc LSTM

So sánh:

Property	RNN	LSTM	GRU
Recurrent	v	v	v
Hidden state	v	v	v
Separate cell state		v	
Gates		v	v
Architecture	Simple	More complex	Simpler than LSTM
Long dependency handling	Limited	Improved	Improved

Không yêu cầu sinh viên tự implement LSTM/GRU từ đầu.

---

19. Experiment 4 — RNN vs LSTM vs GRU

Xây dựng ba model:

```
Model A
Embedding
 ↓
RNN
 ↓
Linear
```

```
Model B
Embedding
 ↓
LSTM
 ↓
Linear
```

```
Model C
Embedding
 ↓
GRU
 ↓
Linear
```

Giữ các điều kiện càng giống nhau càng tốt:

- vocabulary;
- embedding dimension;
- batch size;
- optimizer;
- learning rate;
- number of epochs;
- train/validation/test split.

Chỉ thay đổi recurrent architecture.

Bảng kết quả:

Model	Parameters	Training time	Accuracy	Precision	Recall	F1
RNN						
LSTM						
GRU						

Câu hỏi chính:

> Kết quả thực nghiệm có phù hợp với prediction ban đầu không?

Không được kết luận chỉ dựa vào accuracy.

---

20. Experiment 5 — Sequence length

Giữ architecture cố định.

Thay đổi:

```
max_length ∈ {50, 100, 200}
```

Theo dõi:

- percentage of tokens retained;
- training time;
- validation loss;
- test F1;
- memory usage nếu có thể đo.

Vẽ:

```
max_length
    ↓
performance
```

và:

```
max_length
    ↓
```

```
training time
```

Trả lời:

Có tồn tại trade-off giữa context và computational cost không?

---

21. Experiment 6 — Long-range dependency

Tạo hoặc sử dụng các câu có từ khóa quan trọng ở những vị trí khác nhau.

Ví dụ:

```
"The movie was, despite many unnecessary scenes,
ultimately excellent."
```

và:

```
"The movie was, despite many unnecessary scenes,
ultimately terrible."
```

So sánh các trường hợp trong đó từ quyết định sentiment xuất hiện:

```
near the beginning
near the middle
near the end
```

Mục tiêu:

Quan sát xem vị trí của thông tin quan trọng ảnh hưởng thế nào tới prediction.

Sinh viên không được tuyên bố rằng experiment nhỏ này chứng minh đầy đủ vanishing gradient.

Phải phân biệt:

```
empirical observation
```

với:

```
theoretical explanation
```

---

22. Evaluation

Sinh viên phải báo cáo:

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Precision

$$Precision = \frac{TP}{TP + FP}.$$

Recall

$$Recall = \frac{TP}{TP + FN}.$$

F1

$$F1 = 2\frac{Precision \cdot Recall}{Precision + Recall}.$$

Ngoài classification metrics, phải báo cáo:

- number of parameters;
- training time;
- number of epochs;
- sequence length;
- vocabulary size.

Mục đích là để sinh viên bắt đầu nhận ra:

> NLP model không chỉ được đánh giá bằng predictive performance.

---

23. Learning curves

Vẽ:

```
training loss
validation loss
```

theo epoch.

Ví dụ:

Epoch	Train Loss	Validation Loss
1		
2		
3		
...		

Sinh viên phải trả lời:

> Model có dấu hiệu overfitting không?

Một trường hợp điển hình:

```
Train loss
    ↓↓↓↓↓

Validation loss
    ↓↓ → ↑↑
```

có thể là dấu hiệu model bắt đầu overfit.

Không được chỉ nhìn epoch cuối cùng.

---

24. Error analysis

Chọn ít nhất:

```
3 correctly classified
3 incorrectly classified
```

Bảng:

Text	True	Predicted	Analysis
…	positive	positive	…
…	negative	positive	…
…	positive	negative	…

Với mỗi lỗi, xem xét:

```
negation
long-range dependency
sarcasm
ambiguous sentiment
rare word
domain-specific term
very long sequence
truncation
noise
```

Đặc biệt yêu cầu sinh viên xác định:

> Lỗi này có phải do sequence modeling hay không?

Ví dụ:

```
"I expected the movie to be bad,
but it was actually wonderful."
```

Nếu model sai, sinh viên phải xem xét:

- negation;
- contrast;
- sequence length;
- vocabulary;
- training data;
- hoặc nguyên nhân khác.

Không được mặc định:

> "LSTM không hiểu câu."

---

25. So sánh với LAB 04

Sinh viên quan sát bảng, dựa vào kinh nghiệm đã tổng kết được:

Property	TF-IDF + LR	Avg Word2Vec + LR	RNN	LSTM	GRU
Sparse representation	v				
Dense representation		v	v	v	v

Property	TF-IDF + LR	Avg Word2Vec + LR	RNN	LSTM	GRU
Word order	Limited	x	v	v	v
Context-dependent state	x	x	v	v	v
Long dependency	x	x	Limited	Improved	Improved
Computational cost	Low	Low	Higher	Higher	Higher
One vector per word		v	Embedding	Embedding	Embedding

Sau đó trả lời:

RNN/LSTM/GRU giải quyết được vấn đề gì mà Average Word2Vec không giải quyết tốt?

Và:

RNN/LSTM/GRU vẫn còn vấn đề gì?

---

26. Application

Xây dựng một sentiment classifier hoàn chỉnh:

```
Raw review
 ↓
Tokenizer
 ↓
Vocabulary
 ↓
Token IDs
 ↓
Padding / truncation
 ↓
Embedding
 ↓
LSTM / GRU
 ↓
Linear layer
 ↓
Sentiment
```

Input:

```
"I really enjoyed this movie."
```

Output:

```
Positive
Probability: ...
```

Sinh viên phải thử ít nhất:

- một review ngắn;
- một review dài;
- một review có negation;
- một review có contrast.

Ví dụ:

```
"I thought the movie would be terrible,
but it was surprisingly good."
```

Không chỉ báo cáo prediction.

Phải giải thích:

Những từ hoặc đoạn context nào có khả năng ảnh hưởng tới prediction?

---

28. Individual learning check

Giảng viên random một câu trong 3 phút.

Ví dụ:

RNN khác Average Word2Vec ở điểm nào?

hoặc:

Hidden state (h_t) có ý nghĩa gì?

hoặc:

Tại sao RNN có thể gặp vanishing gradient?

hoặc:

Forget gate của LSTM dùng để làm gì?

hoặc:

Tại sao tăng sequence length có thể làm training chậm hơn?

hoặc:

GRU khác LSTM ở điểm nào?

hoặc:

Tại sao model có training loss giảm nhưng validation loss tăng?

hoặc:

Nếu một review bị truncate ở 30 tokens, thông tin gì có thể bị mất?

---

29. Deliverables

```
lab05/
├── README.md
├── calculations.md
├── prediction.md
├── rnn_cell.py
├── sequence_classification.ipynb
├── results.csv
├── learning_curves.png
├── confusion_matrix.png
├── error_analysis.md
└── reflection.md
```

---

30. Rubric

Thành phần	Điểm
Theory & calculation	10
Prediction	10
Dataset & sequence analysis	10
RNN cell implementation	15
RNN/LSTM/GRU experiment	20
Sequence length experiment	10
Quantitative evaluation	10
Error analysis	5
Reflection	5
Individual learning check	5
Tổng	**100**

---

31. Peer assessment

Reviewer kiểm tra:

```
Dataset
[ ] Dataset được mô tả
[ ] Class distribution được báo cáo
```

```
[ ] Sequence length được phân tích
[ ] Padding/truncation được giải thích

Implementation
[ ] RNN cell được implement
[ ] Forward computation đúng
[ ] Baseline có kết quả
[ ] RNN/LSTM/GRU được so sánh

Experiment
[ ] Có sequence length experiment
[ ] Có training/validation curves
[ ] Metrics đầy đủ
[ ] Computational cost được báo cáo

Analysis
[ ] Confusion matrix
[ ] Ít nhất 3 lỗi được phân tích
[ ] Có phân tích context/sequence
[ ] Không chỉ báo cáo accuracy

Understanding
[ ] Giải thích được hidden state
[ ] Giải thích được LSTM gates
[ ] Giải thích được vanishing gradient
```

---

32. Mạch nối LAB 01 → LAB 05

Đến đây curriculum trở thành:

```
LAB 01
Text
 ↓
TF-IDF
 ↓
Sparse representation
 ↓
Search
```

```
LAB 02
Text
 ↓
N-gram
 ↓
Probability
 ↓
Language Model
 ↓
Perplexity
```

```
LAB 03
Text
 ↓
Context
 ↓
Word Embedding
 ↓
Semantic Similarity
```

```
LAB 04
Text
 ↓
Representation
 ↓
Classifier
 ↓
Prediction
 ↓
Evaluation
 ↓
Error Analysis
```

```
LAB 05
Text
 ↓
Token sequence
 ↓
Embedding
 ↓
RNN
 ↓
Hidden state
 ↓
LSTM / GRU
 ↓
Classification
 ↓
Error Analysis
```

LAB 04 cho thấy:

$$\boxed{\text{Static representation} \neq \text{sequence modeling}}$$

LAB 05 cho thấy:

$$\boxed{h_t = f(x_t, h_{t-1})}$$

tức model bắt đầu duy trì **state theo sequence**.

Nhưng recurrent processing vẫn có một vấn đề lớn:

```
x1 → x2 → x3 → ... → x100
```

Muốn xử lý (x_{100}), thông tin phải đi qua rất nhiều bước recurrent.

Điều này dẫn tới câu hỏi tiếp theo:

> **Có cách nào để một token có thể trực tiếp "nhìn" hoặc truy cập những token quan trọng khác trong sequence mà không phải truyền toàn bộ thông tin qua một chuỗi recurrent dài?**

Đây chính là động lực để chuyển sang:

```
LAB 06
Sequence
 ↓
Attention
 ↓
Query / Key / Value
```

```
 ↓
Context-aware representation
```

và sau đó:

```
LAB 07
Attention
 ↓
Self-Attention
 ↓
Transformer
```

Do đó, LAB 05 không biến RNN/LSTM/GRU thành điểm kết thúc. **Giá trị lớn nhất của lab là để sinh viên tự trải nghiệm cả sức mạnh lẫn giới hạn của recurrent sequence modeling, từ đó thấy được tại sao Attention xuất hiện.**